BOOK & BLOOM — ONLINE BOOKSTORE MANAGEMENT SYSTEM

UML 2.5 Detailed Architecture & Object-Oriented Domain Specification

SLIIT · SE1020 Object Oriented Programming Prorata Project · Three-Tier Architecture (JSP / Servlet / Spring Boot / MySQL & Atomic File Storage)

UML 2.5 NOTATION & RELATIONSHIP LEGEND

- Generalization / Inheritance (`extends`)
- Interface Realization (`implements`)
- Composition (*Order owns OrderLine*)
- Navigable Association (*Multiplicity: 1 to 0..**)

Model Entity · Service Layer · Repository Layer · Servlet Layer

All persisted fields are **private (-)** with corresponding public getters/setters (+).

1. PRESENTATION & CONTROLLER TIER (WEB LAYER)

<<controller>> PageController
- root: Path
- isDev: boolean
+ home(): String
+ catalog(): String
+ book(): String
+ cart(): String
+ orders(): String
+ admin(): String
+ account(): String

<<HttpServlet>> ApiServlet
- mapper: ObjectMapper
- users: UserService
- books: BookService
- authors: AuthorService
- categories: CategoryService
- orders: OrderService
- reviews: ReviewService
+ service(req, res): void
- authenticate(req): String
- validateCsrf(req): void
- parseBody(req): JsonNode

2. BUSINESS LOGIC & SERVICE TIER (6 MEMBERS ALLOCATED CRUD SERVICES)

<<service>> UserService
- repository: StoreRepository
+ authenticate(JsonNode): User
+ register(JsonNode): User
+ profile(String): User
+ updateProfile(String, JsonNode): User
+ delete(String, String): void

<<service>> BookService
- repository: StoreRepository
+ list(): List<Book>
+ save(actor, id, body): Book
+ delete(actor, id): void

<<service>> AuthorService
- repository: StoreRepository
+ list(): List<Author>
+ save(actor, id, body): Author
+ delete(actor, id): void

<<service>> CategoryService
- repository: StoreRepository
+ list(): List<Category>
+ save(actor, id, body): Category
+ delete(actor, id): void

<<service>> OrderService
- repository: StoreRepository
+ create(actor, body): Order
+ updateStatus(actor, id, status): Order
+ delete(actor, id): void

<<service>> ReviewService
- repository: StoreRepository
+ list(): List<Review>
+ submit(actor, body): Review
+ delete(actor, id): void

<<utility / domain rule>> DeliveryFeePolicy
+ FREE_THRESHOLD: BigDecimal = 5000
+ STANDARD_FEE: BigDecimal = 350
+ calculate(subtotal, shipping): BigDecimal

3. CORE DOMAIN MODEL TIER (OOP OBJECT MODEL, INHERITANCE, ENCAPSULATION & ASSOCIATIONS)

<<abstract>> User
- id: String
- name: String
- email: String
- passwordHash: String
- address: String
- createdAt: String
+ getId(): String
+ getName(): String
+ getEmail(): String
+ role(): String*
+ canManageCatalog(): boolean*
+ publicProfile(): Map<String, Object>

<<entity>> Customer
+ role(): String {return "CUSTOMER"}
+ canManageCatalog(): boolean {false}

<<entity>> AdminUser
+ role(): String {return "ADMIN"}
+ canManageCatalog(): boolean {true}

<<abstract>> Book
- id: String
- title: String
- authorId: String
- categoryId: String
- isbn: String
- price: BigDecimal
- stock: int
- description: String
- coverColor: String
- featured: boolean
+ getId(): String
+ getTitle(): String
+ getPrice(): BigDecimal
+ getStock(): int
+ requiresShipping(): boolean*
+ fulfillment(): String*

<<entity>> PrintedBook
- pages: int
+ getPages(): int
+ requiresShipping(): boolean {true}
+ fulfillment(): String

<<entity>> EBook
- fileFormat: String
+ getFileFormat(): String
+ requiresShipping(): boolean {false}
+ fulfillment(): String

<<entity>> Author
- id: String
- name: String
- bio: String
- nationality: String
- primaryGenre: String
- accolades: String
- bornYear: String
- photoUrl: String
+ getId(): String
+ getName(): String
+ getNationality(): String
+ getPrimaryGenre(): String
+ getAccolades(): String
+ getBornYear(): String
+ getPhotoUrl(): String

<<entity>> Category
- id: String
- name: String
- description: String
+ getId(): String
+ getName(): String
+ getDescription(): String

<<entity>> Order
- id: String
- userId: String
- lines: List<OrderLine>
- subtotal: BigDecimal
- deliveryFee: BigDecimal
- total: BigDecimal
- status: String
- address: String
- createdAt: String
+ getId(): String
+ getLines(): List<OrderLine>
+ getTotal(): BigDecimal
+ getStatus(): String
+ setStatus(String): void

<<value object (composite)>> OrderLine
- bookId: String
- title: String
- unitPrice: BigDecimal
- quantity: int
- shipping: boolean
+ getBookId(): String
+ getQuantity(): int
+ getUnitPrice(): BigDecimal
+ isShipping(): boolean

<<entity>> Review
- id: String
- userId: String
- bookId: String
- rating: int (1..5)
- comment: String
- createdAt: String
+ getId(): String
+ getRating(): int
+ getComment(): String
+ getUserId(): String
+ getBookId(): String

<<cryptographic utility>> Passwords
- ALGORITHM: String = "PBKDF2WithHmacSHA256"
- ITERATIONS: int = 210000
- SALT_BYTES: int = 16
+ hash(char[] password): String
+ verify(char[] password, String storedHash): boolean

4. DATA PERSISTENCE & REPOSITORY TIER (ABSTRACTION BEHIND StoreRepository INTERFACE)

<<aggregate state snapshot>> StoreState
- schemaVersion: int = 1
- books: List<Book>
- authors: List<Author>
- categories: List<Category>
- users: List<User>
- orders: List<Order>
- reviews: List<Review>
+ getBooks(): List<Book>
+ getAuthors(): List<Author>
+ getCategories(): List<Category>
+ getUsers(): List<User>
+ getOrders(): List<Order>
+ getReviews(): List<Review>

<<interface>> StoreRepository
+ snapshot(): StoreState
+ transaction(Function<StoreState, T> action): T

<<atomic file storage>> FileStoreRepository
- mapper: ObjectMapper
- file: Path
- lockChannel: FileChannel
- processLock: FileLock
- state: StoreState
+ snapshot(): StoreState
+ transaction(Function): T
+ close(): void
- copy(StoreState): StoreState

<<relational persistence (MySQL 8.0)>> MysqlStoreRepository
- mapper: ObjectMapper
- url: String = "jdbc:mysql://127.0.0.1:3306/book_bloom"
- credentials: Properties
+ snapshot(): StoreState
+ transaction(Function): T
- load(Connection, boolean): StoreState
- persist(Connection, StoreState, StoreState): void
- upsert(Connection, table, columns, values): void

SLIIT · FACULTY OF COMPUTING · SE1020	
PROJECT TITLE: **Book & Bloom (Online Bookstore)**	DRAWING TITLE: **A3 Detailed UML Class Diagram**
PAPER SIZE: **ISO A3 Landscape (420 x 297 mm)**	EVALUATION DATE: **November 2026 (Final Viva)**
ARCHITECTURE: **Three-Tier MVC / Repository Pattern**	STATUS: **100% Implemented & Verified Live**

GROUP WORKLOAD & COMPONENT ALLOCATION (6 TEAM MEMBERS):

IT25102131: User Management & Auth (Customer / AdminUser)
IT25102130: Order & Delivery Management (Order / OrderLine)
IT25102132: Category Management (Navigation & Filtering)
IT25102114: Book Management (PrintedBook / EBook)
IT25101966: Author Management (Era, Awards, Nationality)
IT25102519: Review & Rating Management (Moderation)

GitHub: https://github.com/IT25102131/book-and-bloom-se1020 · Sheet 1 of 1 · Master Evaluation Copy